

你查看一下gith up上的new API这个项目。我想要把这个项目部署到一个海外服务器，作为大模型语言的中转站。而且这个服务器要随着客户越来越多，要能够有扩容扩量的能力。告诉我最好的方法是什么？推荐什么样的服务器？在哪里见我主要服务的客户在西欧，比利时，法国，德国。荷兰

已思考 1m 3s >

我看的是 GitHub 上的 **QuantumNous/new-api**。这个项目本质上是一个统一的 LLM 网关，支持把多家模型接口统一成 OpenAI / Claude / Gemini 兼容格式；仓库当前很活跃，但作者也明确写了它“主要用于个人学习用途，不保证稳定性或技术支持”，而且许可证是 **AGPL-3.0**，商业部署前要先确认你能接受这个许可证约束。

GitHub +1

最好的方法，不是“买一台很大的海外服务器直接跑”，而是按下面这个结构部署：

- 1. **New API 做成无状态应用节点**
- 2. **数据库单独托管**（MySQL / PostgreSQL）
- 3. **Redis 单独托管**
- 4. **前面放负载均衡**
- 5. **以后通过加应用节点横向扩容**

这是因为 New API 官方文档本身就支持集群部署：多台服务器、共享同一个数据库和 Redis、所有节点使用相同的 `SESSION_SECRET` 和 `CRYPTO_SECRET`，再用 Nginx / HAProxy / 云负载均衡把流量分发到各节点。文档还特别说明，多机部署时这两个 secret 必须一致，否则会出现登录态和解密不一致的问题。

New API +2

我给你的结论

如果你要“现在能上、以后能扩、主要服务西欧客户”，最佳方案是：

优先选一个欧洲云厂商，做“托管 Kubernetes + 托管数据库 + 托管 Redis”的架构。

在你列的客户区域里（比利时、法国、德国、荷兰），这比单机 VPS 或单台独服稳得多，也更容易后续扩容。

我最推荐的落地选项

首选：Scaleway（阿姆斯特丹或巴黎）

原因是它在欧洲有 **Paris / Amsterdam / Warsaw** 区域，提供 **Kubernetes Kapsule** 托管 Kubernetes、托管 PostgreSQL/MySQL、托管 Redis，而且 Redis 明确支持横向/纵向扩展与高可用集群。对你这种“网关型服务”很合适，运维复杂度比较低。阿姆斯特丹对比利时、荷兰、德国西部通常延迟很友好；巴黎对法国客户更友好。

Scaleway +4

次选：OVHcloud（法国或法兰克福）

OVHcloud 在法国和德国法兰克福都有基础设施，也提供 **Managed Kubernetes** 和托管 PostgreSQL。它的欧洲本地属性、合规叙事、网络覆盖都不错，适合你如果更重视“欧洲本地云”和企业采购接受度。

OVHcloud +2

预算优先：Hetzner（德国）

如果你很在意成本，Hetzner 很有吸引力，在德国纽伦堡、Falkenstein 和芬兰有自营数据中心。问题是它原生托管数据库能力明显弱一些，官方云产品生态不如 Scaleway / OVHcloud 一体化；这会把更多运维工作压到你自己身上。

hetzner.com +2

具体选哪里

你的客户在 **比利时、法国、德国、荷兰**，我建议这样选：

单区域起步：阿姆斯特丹优先

它对荷兰、比利时、德国西部都比较均衡；如果法国客户占比更高，就选巴黎。Scaleway 现在就有 Amsterdam 和 Paris 两个成熟区域可选。

Scaleway +1

做高可用：主区 Amsterdam，灾备区 Paris 或 Frankfurt

这样能兼顾 Benelux / 德国 / 法国。

如果你选 OVHcloud，则可以用 **Frankfurt + France** 的组合；OVH 官方列出的欧洲点位包括 Paris、Roubaix、Strasbourg 和 Frankfurt。

OVHcloud

不建议的做法

不建议一开始就上“1 台超大机器 + SQLite”。

New API 文档里 SQLite 适合本地或简单部署，但要多机/扩容时，官方要求共享数据库和 Redis。也就是说，只要你真想做规模化，就应该从一开始用 MySQL/PostgreSQL + Redis。

New API +2

你应该怎么配服务器

这是我给你的实用规格建议：

阶段 1：刚上线 / 小规模商用

- 2 个应用节点：每台 **4 vCPU / 8 GB RAM**
- 1 个托管 PostgreSQL 或 MySQL：**2 vCPU / 4–8 GB**
- 1 个托管 Redis：**2–4 GB 内存**
- 1 个负载均衡器
- 1 个对象级监控/日志方案

阶段 2：客户开始增长

- 应用节点扩到 **3–5 台**
- 每台升到 **8 vCPU / 16 GB RAM**，或者保持 4 vCPU 但多加节点
- Redis 开启高可用
- 数据库升级为高可用主备

- 接入 CDN / WAF / DDoS 防护

阶段 3：稳定商用

- Kubernetes 上跑 New API
- HPA 自动扩容
- 蓝绿发布 / 滚动更新
- 主区 + 次区容灾

这里关键点是：**New API 是网关，不是模型推理本体**。所以你通常不需要 GPU，真正重要的是：

- 稳定的公网带宽
- 足够的并发连接
- Redis / DB 的可用性
- 上游模型供应商的限流管理
- 日志、审计、告警

最推荐的实际方案

如果让我直接替你拍板，我会这样选：

方案 A：最均衡，最推荐

- 云厂商：Scaleway
- 区域：Amsterdam
- 架构：Kubernetes Kapsule + Managed PostgreSQL/MySQL + Managed Redis
- 前置：Nginx Ingress 或云 LB
- 适合：你现在就想做正规商用，而且未来要平滑扩容

方案 B：更偏企业/合规叙事

- 云厂商：OVHcloud
- 区域：Frankfurt 或 France
- 架构：Managed Kubernetes + Managed PostgreSQL + Redis/Valkey
- 适合：你希望更强调欧洲本地基础设施和后续企业客户接受度

方案 C：最省钱

- 云厂商：Hetzner
- 区域：德国
- 架构：2-3 台云主机 + 自建 K3s/Kubernetes + 自建 PostgreSQL/Redis
- 适合：你自己 DevOps 很强，能接受更多运维负担

还有两个很重要的提醒

第一，New API 官方虽然支持集群，但文档里也写了项目不保证稳定性/技术支持。所以你最好：

- 自己 fork 一份
- 固定版本，不要直接追 latest
- 上线前压测

- 准备回滚方案

GitHub

第二，AGPL-3.0 要先看清楚。你如果是对外提供商业化中转服务，这个许可证影响不能忽略。 GitHub +1

我的一句话建议是：

不要买“单台海外大机器”；直接在阿姆斯特丹或巴黎，用 Scaleway 的托管 K8s + 托管数据库 + 托管 Redis 部署 New API。

这是你面向西欧客户时，当前最平衡的可扩展方案。

如果你愿意，我可以下一步直接给你一套**可执行的部署蓝图**：包括域名、Nginx/K8s、数据库、Redis、备份、监控、扩容和成本分级配置。

     ...    来源

我自己在源代码的基础上还要做一些更新，做一些其他功能的开发，在我本机上做。你的方案里边要考虑到我后续把网站更新过后要能够直接推送到服务器端。让客户能够看到更新的功能。把这个角度考虑进去过后，哪一个方案才是最优的？尤其是要考虑到扩容的方便。我对你说的第三个方案很感兴趣，给我仔细介绍一下。

已思考 27s >

把“你要在本机持续改源码、加功能、然后能稳定推到线上，并且后面还要方便扩容”这个条件加进来以后，结论会变：

如果你偏重成本、控制权、自己能动手运维，那么你感兴趣的第三个方案可以变成最优方案。
但要注意，这个“最优”是指：**在性价比、自主可控、后续代码持续发布、可扩容**这几个目标一起考虑时最优；不是“运维最省心”最优。运维最省心仍然是托管 Kubernetes 那条路。

对你这个场景，我会把第三个方案升级成下面这个版本：

最优的第三方案：Hetzner Cloud + GitHub + Docker + K3s + GitHub Actions

不是“本机改完代码后手工 rsync 上传”，也不是“单机 docker-compose 一把梭”。
而是：

本机开发 → 推到 GitHub 私有仓库 → GitHub Actions 自动构建镜像 → 推到镜像仓库 → Hetzner 上的 K3s 集群自动滚动更新

这样做的好处是：

- 1. 你本机开发最自由，直接改你 fork 后的 new-api 源码。
- 2. 上线动作标准化，不是每次 SSH 上服务器手改。
- 3. 扩容方便，后面加节点、加副本、加负载均衡都比较顺。
- 4. 回滚容易，某个版本有问题，回滚到上一个镜像 tag 就行。
- 5. 多服务器一致性好，不会出现“这台改了那台没改”的情况。

New API 官方文档本身就支持 Docker Compose，也支持集群部署；集群部署要求多个节点共享同一个数据库和 Redis，并且 `SESSION_SECRET`、`CRYPTO_SECRET` 必须一致。也就是说，它天然适合做成“无状态应用副本 + 外部 DB/Redis”的模式。

New API +1

为什么现在第三方案会比“单机部署”更适合你

因为你已经明确说了两件事：

- 你要继续在源码基础上做二次开发
- 以后客户会越来越多，要扩容

这两个条件一加，最怕的就是：

你把线上服务器当开发机用。

那样短期快，长期很乱。
一旦你有 2 台、3 台、5 台服务器，就容易出现版本不一致、回滚麻烦、配置漂移、上线靠手工记忆的问题。

所以第三方案里，真正核心的不是 Hetzner 本身，而是这条发布链路：

Git 版本管理 + 容器镜像 + 自动部署 + 多实例负载均衡

我对第三方案的正式建议

方案定位

Hetzner Cloud 上跑一个轻量级 K3s 集群，前面用 Hetzner Load Balancer，后面用独立的 PostgreSQL/MySQL 和 Redis。应用本身用你二次开发后的 New API 镜像。Hetzner 官方提供私有网络、负载均衡和 API，适合这种结构；它在欧洲的自营数据中心位于德国纽伦堡、Falkenstein 和芬兰赫尔辛基，面向西欧客户部署在德国区域通常比较合适。

docs.hetzner.com +3

最推荐的区域

对你服务的客户群（比利时、法国、德国、荷兰），我建议优先放在 Hetzner 德国区。
原因很简单：地理上更均衡，且 Hetzner 在德国自有自营数据中心。官方列出的欧洲自营数据中心是 Nuremberg、Falkenstein、Helsinki。

docs.hetzner.com +1

在 Hetzner 里我更建议你：

- **首选** Nuremberg
- **备选** Falkenstein
- **不优先** Helsinki，因为你客户主要在西欧大陆，不需要把主站放到更北边

这套架构长什么样

建议你按这个结构来：

第 1 层：入口层

- Hetzner Load Balancer
- 域名 + TLS
- 只暴露 80/443

第 2 层：应用层

- 2~3 台 Hetzner Cloud Server
- 上面组成一个 K3s 集群
- 跑你修改后的 New API 容器
- 应用做成多副本 Deployment

第 3 层：数据层

- 1 个 PostgreSQL / MySQL
- 1 个 Redis
- 定时备份
- 不和应用混在一起

第 4 层：私网

- 所有应用节点和数据库走 Hetzner Private Network
- 应用节点尽量不直接对公网暴露
- 只让 Load Balancer 对外

Hetzner 官方文档明确支持私有网络，服务器之间可通过私网通信；官方社区近期开源教程里也给出了“公网负载均衡 + 私网应用节点 + NAT”的生产式结构，这和你这种 API 网关业务很贴近。

[docs.hetzner.com](#) +1

为什么我不建议你从 docker-compose 单机开始

可以用，但不建议作为长期方案。

因为如果你先这样做：

- 本机改代码

- build
- ssh 到服务器
- docker-compose pull / up -d

短期确实能跑。

但一旦你开始加第二台机器，复杂度会突然上来：

- 哪台先更新
- 版本是否一致
- LB 怎么摘流量
- 迁移时会不会中断
- 回滚怎么做
- 配置文件怎么同步

所以如果你已经明确说“以后肯定会扩”，那我建议你**直接从 K3s 起步**。

K3s 比完整 Kubernetes 轻很多，但已经足够支撑：

- 滚动更新
- 多副本
- 自动重启
- 基本服务发现
- 后续水平扩容

最适合你的开发发布流程

这是我认为对你最顺手的一条链路。

本地开发

你在自己电脑上：

- fork `QuantumNous/new-api`
- 新建你自己的私有仓库
- 在本机开发、测试
- 每个功能走 branch
- 合并到 `main` 或 `release`

New API 官方也有本地开发和 Docker Compose 文档，所以你在本机把它跑起来做二开是顺路的。

New API +1

持续集成

GitHub Actions 自动做这些事：

1. 检出代码
2. 构建 Docker 镜像
3. 打 tag，比如 `v1.3.7` 或 `git-sha`

4. 推到镜像仓库

镜像仓库你有三个可选：

- GitHub Container Registry
- Docker Hub
- 自建 Harbor

我更建议 **GHCR**，因为你本来就用 GitHub 管代码，链路最短。

持续部署

部署有两种方式。

方式 A：GitHub Actions 直接部署到 K3s

优点是简单。

就是 workflow 在镜像 push 完后，直接 `kubectl apply` 或 `helm upgrade` 。

方式 B：GitOps (Argo CD / Flux)

优点是更规范，后期更稳。

流程变成：

- GitHub Actions 只负责构建镜像
- 更新 Helm values 里的镜像 tag
- Argo CD 自动把变更同步到集群

对你这种“后续还会持续加功能”的项目，**我更推荐** GitOps。

因为它的上线历史、回滚、环境一致性都更干净。

这套方案的扩容方式

这是第三方案最大的价值。

应用扩容

最简单。

客户多了以后，你只需要把 New API 的副本数从 2 改成 4、6、8。

Hetzner 的负载均衡本来就是为多目标节点分流设计的；官方说明也明确写了负载均衡用来把流量分配到多个目标服务器，以提高可用性、扩展性和性能。[docs.hetzner.com +1](#)

节点扩容

如果单机 CPU / 内存不够，就在 Hetzner 再加云主机，把它加入 K3s 集群。

由于 Hetzner 有私有网络和 Cloud API，这件事可以自动化。官方 API 和基础设施文档都支持这类程序化管理。[docs.hetzner.com +1](#)

数据层扩容

这部分要比应用层谨慎。

- Redis 可以先单实例，再做主从或替换为托管/高可用方案
- 数据库可以先单机，再迁到独立更大实例

注意，**New API 应用层是最容易横向扩的，数据库不是。**

所以初期预算不高时，最省钱的办法是：

- 先把应用层按“可横向扩”设计好
- 数据库先保守做
- 等客户量起来后再升级 DB/Redis

你最该怎么选：K3s 还是 Docker Compose

我给你直接定：

如果你只是测试市场、客户数很少、1 个月内不考虑扩容

可以先：

- 1 台应用机
- 1 台数据库/Redis 机
- docker-compose 部署

但如果你已经明确说“后续会持续开发并且要扩容”

那就直接：

- 2 台 K3s worker/control-plane 组合
- 1 台独立 DB/Redis
- 1 个 Hetzner LB
- GitHub Actions 自动发布

对你这种场景，**这才是最优解。**

我给你的推荐配置

起步版

适合刚上线。

- 2 台应用节点：每台 4 vCPU / 8 GB
- 1 台 DB/Redis：8 vCPU / 16 GB
- 1 个 Hetzner Load Balancer
- 1 个 Private Network

- 对象存储做备份

成长期

客户开始多时。

- 3~4 台应用节点：每台 4 或 8 vCPU / 8–16 GB
- 数据库独立强化
- Redis 独立强化
- 监控和日志加上

稳定商用版

- 3 台以上 K3s 节点
- DB 和 Redis 分离
- GitOps
- 蓝绿或滚动发布
- 异地备份

第三方案的优点和代价

优点

第一，代码上线最顺。

你本机改完后，只需要 push。后面的 build、发版、更新，都能自动完成。

第二，扩容很自然。

不是“换一台更大的机器”，而是“加副本、加节点”。

第三，成本很漂亮。

Hetzner 的云服务一直以高性价比著称，官方也强调其价格性能比优势。[hetzner.com](#) +1

第四，欧洲部署位置合适。

德国区对你这批西欧客户很合理。Hetzner 也强调 GDPR 合规和德国/芬兰数据中心的安全认证。[hetzner.com](#)

代价

第一，运维复杂度高于 Scaleway/OVH 托管 K8s。

Hetzner 官方并没有原生托管 Kubernetes 产品页面；现有资料更多是社区方案、第三方 Cloudfleet，或自建 Kubernetes/Talos/K3s。[community.hetz...](#) +2

第二，数据库和 Redis 你要自己管，或者自己选替代服务。

这比全托管云更费心。

第三，区域切换和资源规划要提前想好。

Hetzner 的网络、负载均衡、卷等资源和 location / network zone 绑定较紧，后面迁移不如托管大云灵活。这个在官方和社区资料里都能看出来。

docs.hetzner.com +2

最后的明确结论

把“本机持续改源码 → 一键推到服务器 → 客户马上看到 → 后面容易扩容”这个要求纳入后：

总体上

如果你想最省心，最优还是托管 K8s。

但如果你明确偏向第三方案

那第三方案的最优形态不是“Hetzner 单机”，而是：

Hetzner Cloud + K3s + GitHub Actions + Docker Registry + Hetzner Load Balancer + 独立 DB/Redis

这是我认为最适合你的版本。

一句话说，就是：

你要把“服务器”当运行环境，不要当开发环境；把“更新”做成流水线，不要做成手工上传。

如果你要，我下一步可以直接给你出一份可落地的第三方案实施图：

包括目录结构、GitHub Actions workflow、K3s 部署方式、域名/TLS、数据库拆分，以及第一阶段最省钱的 Hetzner 机器规格。

     ...    来源